SECURITY

XSS Prevention Visual Guide

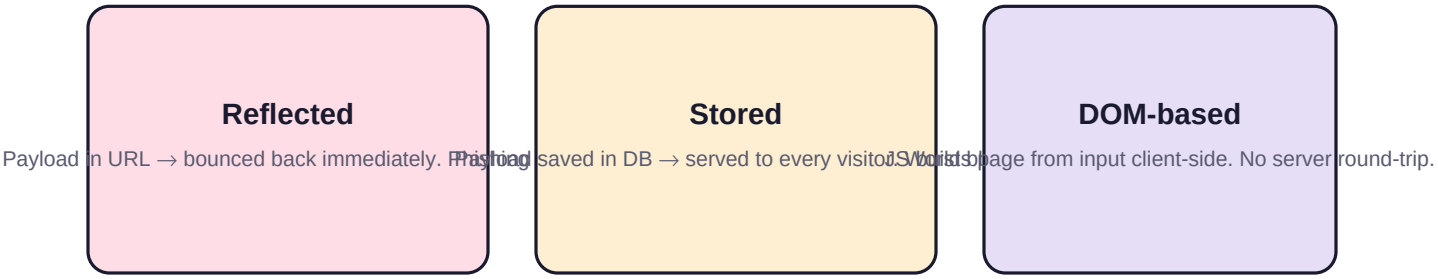
Cross-Site Scripting —
stop attacker scripts running in your users' browsers.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Three XSS Types

Same payload, three delivery routes.



STEP 2/6

Output Encoding by Context

Escape for WHERE the data lands, not just that it exists.

HTML body

escape < > & quotes

Attribute

quote-encode

JavaScript

\u-encode

HTML-encoding inside a script block does nothing — the browser still runs it. Context-aware encoding (or a framework that does it) is the fix.

STEP 3/6

Content-Security-Policy

A browser-enforced allowlist for what may execute.

Content-Security-Policy: default-src 'self'; script-src 'self'

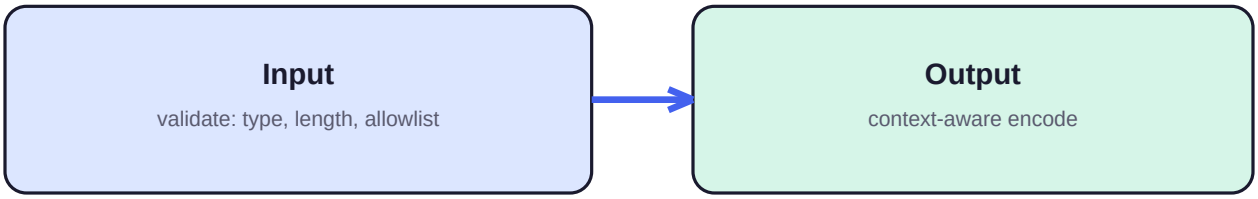
header sent with every page

- Blocks inline scripts and unknown origins — kills most XSS payloads even if injected.
- Start with report-only mode; fix violations before enforcing.
- Add nonces/hashes for the few inline scripts you truly need.

STEP 4/6

Input Handling Done Right

Validate on the way in, encode on the way out.



Store raw input, encode at render time. Encoding at storage corrupts data and still misses contexts.

STEP 5/6

Cookie Defenses

Shrink what a successful XSS can steal.

- HttpOnly: cookies invisible to JavaScript — XSS can't read the session token.
- Secure: cookies only over HTTPS. SameSite=Lax/Strict: not sent on cross-site requests.
- Prefer short-lived tokens + refresh rotation over giant session cookies.
- These don't stop XSS — they limit it to actions, not full account theft.

STEP 6/6

Frameworks & Testing

Let the framework do the escaping — then verify.

- Modern frameworks auto-escape by default — danger only at raw-HTML APIs (innerHTML, dangerouslySetInnerHTML).
- Audit every raw-HTML escape hatch; sanitize with a proven library if user HTML is required.
- DAST scans and manual testing with polyglot payloads across all three XSS types.
- Monitor CSP violation reports — they're an early warning radar for injection attempts.

STEP ✓/✓

Key Takeaways

XSS — Remember

- Three types: reflected (URL), stored (database), DOM-based (client JS).
- Encode output for its context — HTML, attribute, and JS need different escaping.
- Content-Security-Policy blocks injected scripts at the browser level.
- Validate input, encode at render time; never encode at storage.
- HttpOnly + Secure + SameSite cookies contain the damage.
- Avoid raw-HTML APIs; test with DAST and watch CSP reports.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev